29. Cache Mekanizmaları: Redis ile Yıldırım Hızında Yanıtlar

Cache-Aside Deseni ve ‘Cache Invalidation’ Zorluğunu Aşmak
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *“‘Bilgisayar bilimlerinde sadece iki zor şey vardır: Cache Invalidation ve isimlendirme.’ Peki Redis ile önbellek yönetirken bayat (stale) veriyi nasıl önleriz?”*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 En hızlı veritabanı sorgusu, hiç yapılmayan veritabanı sorgusudur; sık okunan veriler RAM’de (Redis) tutulmalıdır. - 📌 Cache-Aside deseni: Önce Redis’e bak (Cache Hit), yoksa veritabanından oku ve Redis’e yaz (Cache Miss). - 📌 Her önbellek anahtarının mutlaka bir yaşam süresi (TTL - Time to Live) olmalıdır; sonsuz önbellek bellek patlamasına yol açar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir kütüphane görevlisini hayal edin.

Günde 500 öğrenci gelip aynı kitabı soruyor: *“Harry Potter 1. kitap var mı?”*

Eğer kütüphaneci her seferinde 3 kat yukarı çıkıp, tozlu rafların arasında 15 dakika yürüyüp kitabı kontrol edip gelseydi akşamı zor ederdi! (Veritabanı Disk Erişimi).

Akıllı bir kütüphaneci ne yapar? O çok sorulan kitabın bir kopyasını hemen masasının çekmecesine koyar! Biri sorduğunda çekmeceyi açıp 2 saniyede gösterir (Redis RAM Belleği).

Ancak bir kural vardır: Eğer kitap kütüphaneden atılırsa veya güncellenirse, çekmecedeki kitabı da hemen yenilemek gerekir (Cache Invalidation).

🧠 2. AI Çağında Yeni Zihniyet Modeli

Cache-Aside Deseni Akışı:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Dağıtık Sistemler ve Cache Mimarı
Node.js ve Redis (ioredis) kullanarak bir 'Ürün Detay Önbellek Servisi' yazacağız.

## GÖREV:
1. 'Cache-Aside' desenini uygulayan tip güvenli bir fonksiyon yaz.
2. Ürün verisini Redis'te saklarken 1 saatlik TTL (Time to Live) ekle.
3. Ürün güncellendiğinde (`updateProduct`) Redis'teki eski anahtarı anında silen
(Invalidation) fonksiyonu hazırla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü Cache: TTL yok (sonsuza kadar bellekte kalır) ve güncellemede silinmiyor
(bayat veri)
export async function getProductNaive(id: string) {
  const cached = await redis.get(`product:${id}`);
  if (cached) return JSON.parse(cached);

  const product = await db.product.findUnique({ where: { id } });
  // TTL YOK! Bellek dolunca Redis çöker; ürün fiyatı değişse de eski fiyat kalır!
  await redis.set(`product:${id}`, JSON.stringify(product));
  return product;
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import Redis from "ioredis";

const redis = new Redis(process.env.REDIS_URL || "redis://localhost:6379");
const CACHE_TTL_SECONDS = 3600; // 1 Saat

export class ProductCacheService {
  private static getKey(id: string): string {
    return `cache:product:${id}`;
  }

  // 1. Cache-Aside Okuma
  static async getProduct(id: string): Promise<Product | null> {
    const key = this.getKey(id);

    try {
      const cached = await redis.get(key);
      if (cached) {
        return JSON.parse(cached) as Product; // Cache Hit (1 ms)
      }
    } catch (err) {
      console.warn("Redis okuma hatası, DB'ye yönlendiriliyor:", err);
    }

    // Cache Miss: Veritabanından oku
    const product = await db.product.findUnique({ where: { id } });
    if (!product) return null;

    // Redis'e TTL ile kaydet (Fail-safe)
    try {
      await redis.set(key, JSON.stringify(product), "EX", CACHE_TTL_SECONDS);
    } catch (err) {
      console.error("Redis yazma hatası:", err);
    }

    return product;
  }

  // 2. Cache Invalidation (Güncelleme Anında Temizleme)
  static async invalidate(id: string): Promise<void> {
    try {
      await redis.del(this.getKey(id));
    } catch (err) {
      console.error("Cache invalidation hatası:", err);
    }
  }
}
```

💡 **Kod Analizi:** Kıdemli yaklaşımda; Redis çöktüğünde uygulamanın durmaması için `try/catch` ile sessiz devralma (fallback to DB) sağlanmış, `EX` parametresi ile 1 saatlik yaşam süresi zorunlu kılınmış ve güncelleme anında `del` ile bayat veri riski önlenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Redis'e TTL (Time-To-Live) vermeden anahtar kaydetmek ve sunucu belleğini patlatmak.
  - **Çözüm:** İstisnasız her cache anahtarına makul bir TTL süresi verin.
- **Tuzak:** Redis çöktüğünde tüm backend uygulamasının da çökmesine izin vermek.
  - **Çözüm:** Redis çağrılarını try/catch ile sarın; Redis yoksa doğrudan veritabanına gidin (Graceful Degradation).
- **Tuzak:** Kullanıcıya özel hassas verileri genel (public) cache anahtarlarında saklamak.
  - **Çözüm:** Cache anahtarlarına kullanıcı ID'si ekleyin (`user:123:profile`) ve hassas verileri şifreleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 29: Cache Stratejileri - Redis ile Yıldırım Hızı*

- 🖥️ **Slayt 1:** Neden en hızlı sorgu hiç yapılmayan sorgudur?
- 🖥️ **Slayt 2:** Cache-Aside Deseni: Hit, Miss ve TTL kavramları.
- 🖥️ **Slayt 3:** Cache Invalidation kabusu ve çözüm yöntemleri.
- 🖥️ **Slayt 4:** Graceful Degradation: Redis çökerse sistem nasıl ayakta kalır?

Canlı Kodlama / Ekran Kaydı Senaryosu:
Docker üzerinde yerel bir Redis ayağa kaldırın. Postman ile API'ye istek atın: İlk istekte sürenin 120 ms olduğunu (DB okuma), ikinci istekte 2 ms'ye düştüğünü (Redis Hit) canlı gösterin. Ardından veriyi güncelleyip cache'in nasıl temizlendiğini kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Docker ile yerel makinenizde bir Redis başlatın (`docker run -p 6379:6379 redis:alpine`) ve projenizdeki en sık okunan endpoint'e Cache-Aside deseni ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Önbelleğe alınan bir veriye 'TTL' (Time to Live) süresi tanımlamanın temel amacı nedir? - A) Veriyi şifrelemek - B) Belirtilen süre dolduğunda verinin bellekten otomatik silinmesini sağlayarak hem RAM'in dolmasını engellemek hem de bayat veri riskini sınırlamak - C) Bilgisayarı kapatmak -

D) Sadece Python’da hata vermesini sağlamak > **Doğru Yanıt:** B > **Açıklama:** TTL, anahtarın yaşam süresini belirler; süresi biten anahtar silinir ve RAM’in taşması önlenir.

Soru 2: Cache-Aside deseninde ‘Cache Miss’ ne anlama gelir? - A) Sistemin tamamen çöktüğünü - B) Aranan verinin önbellekte (Redis) bulunamadığını; bu yüzden asıl veritabanından okunup önbelleğe yazılması gerektiğini - C) CSS dosyasının eksik olduğunu - D) Kullanıcının şifresini unuttuğunu > **Doğru Yanıt:** B > **Açıklama:** Cache Miss, aranan bilginin Redis’te olmaması durumudur; uygulama veritabanından veriyi çeker, Redis’e kaydeder ve kullanıcıya döner.